Technologisches Gewerbemuseum

Höhere Technische Lehranstalt für Informationstechnologie
Schwerpunkt Systemtechnik

Diploma Thesis

Digital Twin

Smart City Data Distribution and Integration Network

Implementation of a cloud service for data processing with an efficient big data interface
Arseniy Shukhov 5DHIT

Implementation of a distributed DNS service with an integrated data storage for IoT information
Dejan Plisnic 5DHIT

Implementation of an efficient and reliable load balancing method for distributed DNS networks and IoT nodes
Fabian Lehmann 5DHIT

Implementation of a message queuing system for fast and secure communication services
Jakob Laback 5CHIT

Architecture for scalable and distributed networks using consensus algorithms
Sebastian Bruckner-Hrubesch 5DHIT

Architecture for reliable and manageable networks of sensors and data streaming
Thomas Matschinegg 5DHIT

Teacher: Michael Borko, Bakk.techn.
Realized in 2020/21

Delivery note:
7. April 2021 Accepted by:

Statutory declaration

I declare that I have authored this thesis independently, that I have not used other than the declared sources and that I have explicitly marked all material which has been quoted either literally or by content from the used sources.

Location, Date	Arseniy Shukhov
Location, Date	Dejan Plisnic
Location, Date	Fabian Lehmann
Location, Date	Jakob Laback
Location, Date	Sebastian Bruckner-Hrubesch
Location, Date	Thomas Matschinegg

Contents

1 Restriction notices (= Sperrvermerk) 11

2 Introduction 13

2.0.1 Service Mesh . . . 13

2.0.2 Distributed DNS . . . 13

3 Study 15

3.1 Distributed Systems . . . 15

3.1.1 What is a distributed system? . . . 15

3.2 Conception and implementation of an architecture for scalable and distributed DNS networks using consensus algorithms . . . 16

3.2.1 Introduction . . . 16

3.2.2 Consensus in distributed systems . . . 16

3.2.3 Consensus algorithm . . . 16

3.2.4 Replicated state machine . . . 18

3.2.5 Which consensus algorithm fits the needs for our system? . . . 19

3.2.6 Conclusion . . . 23

Glossary 25

Acronyms 31

Bibliography 33

Chapter 1

Restriction notices (= Sperrvermerk)

Die vorliegende Arbeit enthält Geschäftsgeheimnisse der Accenture GmbH und ist daher streng vertraulich. Sie ist ausschließlich als Prüfungsvorlage bestimmt und darf fremden Dritten nicht zugänglich gemacht werden. Eine Vervielfältigung und Veröffentlichung dieser Arbeit, auch auszugsweise oder in digitaler Form, ist ohne ausdrückliche vorherige Einwilligung der Accenture GmbH nicht erlaubt. Diese Geheimhaltungspflicht ist auf fünf Jahre nach der Einreichung der Arbeit begrenzt.

Chapter 2

Introduction

Our team took part in the Accenture Hackathon 2020 in Vienna and managed to secure a place within the top 5 . It was part of the awarded teams and won a cooperation for the diploma thesis with Accenture. The topic-finding and working process started in early spring of 2020, the official start by school and bmbwf was in September.
Our mission is to make smart city data easily accessible and usable for everyone. Due to that, the software is of course provided as Open-Source-Solution.
Modern Cities produce huge amounts of data. Analyzing and visualizing the gathered data is not possible with open-source software yet.

2.0.1 Service Mesh

The digital twin creates a digital duplicate of smart cities fed with live data. Smart Cities have a variety of different data-sources like water throughput, weather data and power consumption from all the different locations in the city. These sources generate lots of data which is not easy to access and visualize yet. The digital twin collect this data, processes it and generates an easily comprehensible visual representation of it. Sorting and filtering functions provide an even better and easier simplicity and processing.

2.0.2 Distributed DNS

At Digital Twin we strive for great performance even for Smart Cities with thousands of sensors. To achieve this, every part of the system, from the individual sensors to the data collection services, has to respond quickly and be available. This big collection of sensors needs some form of management. No one want's to remember thousands of IP addresses for every sensor by heart. This job is done by our **D**ynamic **N**ame **S**ystem service (short **dns** service) which translates domain names like *www.example.com* to ip addresses like *172.164.194.70*.
The Distributed DNS is a lot more sophisticated than a basic DNS server. Key features like the **distributed architecture** paired with **error detection** and an included **metadata store** are not commonly available in a single non-commercial package on the market. Thanks to the distributed nature of our system it is easily scalable and also fault tolerant.

Chapter 3

Study

After defining the problems and goals, research should be carried out to find out how these can be achieved or solved. This study deals with possible solutions and technologies and analyzes their properties in order to find concrete advantages and disadvantages. This section ends with a conclusion.

3.1 Distributed Systems

In the introduction above, the term "distributed system" is mentioned a couple of times. It is very important because this project features not one, but two distributed systems. Therefore, it is essential to understand what a distributed system is before continue reading.

3.1.1 What is a distributed system?

Tanenbaum's definition of a distributed system is: "A distributed system is a collection of independent computers that appears to its users as a single coherent system".[37, p. 2] That means a distributed system consists of autonomous components. Another aspect is that users think they are dealing with a single system [37], instead it might be distributed over the whole world. These autonomous components, often referred to as *nodes* or *agents*, have to collaborate together. That is the challenge and goal of distributed systems. A distributed system is the opposite of a centralized system, which is just one system or a single point of failure. Though it is way easier to manage, it brings a lot of restrictions and a loss of flexibility. For example, it is not as scalable as a distributed system.

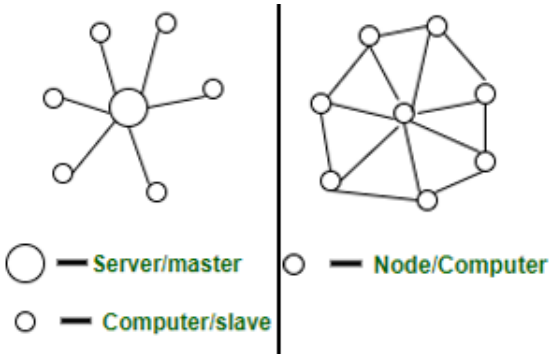


Figure 3.1: Centralized system (left) and distributed system (right) [14]

3.2 Conception and implementation of an architecture for scalable and distributed DNS networks using consensus algorithms

This chapter aims to explain the basic principle of consensus decision making in distributed systems.

3.2.1 Introduction

The system we are building consists of multiple servers. This is called a distributed system, which is further explained in "What is a distributed system?" (3.1.1). Those servers have to replicate each other, which means they are all on the same state and store the same data. Therefore, a user contacting a server receives the same answer no matter what server responds to him. This is called a *Replicated State Machine* and is closely related to *Consistency*. Those topics will also be further explained in later chapters. But how do they manage to stay replicated? They use an algorithm that continuously exchanges information between them, the so-called *Consensus Algorithm*. What it is, how it works and what options are "State of the Art" as of today will be discussed in this section.
But first, to understand the problem, the concept of consensus decision making has to be explained.

3.2.1.1 What is consensus?

To understand the consensus problem in computer systems, it is a good idea to understand basic consensus decision making first. With humans, consensus decision making is used in groups to agree on proposals without one person being strictly against it. In contrast to voting for an item with the majority of the group being happy with it, when using consensus, everyone has to agree on the solution. After everyone consented to a solution, a consensus is made and the group can move forward. [7]
This principle can also be used in the world of computers, more precisely in distributed systems.

3.2.2 Consensus in distributed systems

Consensus means multiple servers agreeing on the same information. It is no problem in single server systems because there is only one party that has to make a decision.
Nevertheless, consensus is a fundamental problem in fault-tolerant distributed systems and multi-agent systems because multiple parties have to agree on proposals. It basically ensures that even if one or more processes have failed, the system consisting of multiple servers still agrees on the same information and makes the correct decisions (fault-tolerance). [41] Consensus is widely used in distributed systems. A very common use of consensus lies in distributed data-stores. When committing a new entry into a database cluster, each node has to decide if they should commit the entry or abort it. If one node wants to abort, everyone has to abort. Other appliances of consensus are for example the synchronization of clocks, electing a leader or *replicated state machines*. In fact, consensus is part of every coordinated decentralized system. [41]

3.2.3 Consensus algorithm

In the consensus problem, a set of agents must all agree on a decision based on their initial states. Typically, only two decisions are allowed, 0 and 1. [41] The decision for 1 may stand for "commit" or simply "yes" and the decision for "0" for "abort" or simply "no". Every agent has to output the same value in order to reach consensus and also there must be some initial state from which 0 is the output and another for which 1 is the output. Formally, a fault-tolerant consensus protocol must have the following properties: [41]

- **Consistency / Agreement**
  All nodes agree on the same value and all decisions are final.
- **Validity / Integrity**
  The agreed-upon value must have been some node's input.
- **Termination**
  Each node decides on a value within a finite number of steps / Every node has to eventually decide some value.

A consensus algorithm derives its properties from the now mentioned characteristics to the following: [31]

- **Safety**: Under all non-Byzantine conditions (what this means will be discussed later), including packet-loss, network delays, etc., they ensure safety **(never returning an incorrect result)**.
- **Availability**: As long as any majority of the servers are operational and able to communicate with each other and clients, they are **fully functional**. Therefore, a cluster of five servers can tolerate the failure of two servers.
- They are **not dependable on timing** to ensure that logs are consistent. Extreme message delays or faulty clocks can, at worst, cause availability problems.
- A minority of slow servers do not impact the overall system performance because a command can complete as soon as a majority of the server cluster has responded to a single round of remote procedure calls.

But what happens if a node is faulty? "A component is faulty once its behavior is no longer consistent with its specification." [33] In consensus algorithms, two categories of failures are the most problematic: **fail-stop failures** and **byzantine failures**.
A fail-stop failure is basically a node not responding. It may have crashed or is not available anymore. A Byzantine failure is a bit more complex. A Byzantine failure is the loss of system agreement due to a Byzantine fault. A fault is considered a Byzantine fault if a node sends back a faulty or malicious message but not the message it is intended to send. It can inconsistently appear both failed and functioning to failure-detection systems, presenting different symptoms to different observers. This leads us to the "Byzantine Generals Problem".

3.2.3.1 Byzantine Generals Problem

The byzantine generals problem describes and illustrates the consensus problem in distributed systems very well.
Imagine a city and a byzantine army which consists of one general and three lieutenants. The general decides that he wants to attack the city and overtake it. He only has one problem: How does he communicate with the lieutenants which are *distributed* around the city? [22]
Basic example
He decides to send a message and employs a messenger to communicate with them and coordinate the attack. This should work really well if everyone is honest. But what if one lieutenant is a traitor and wants the army to lose? He can tell a different message to another lieutenant, for example to retreat. However,

the other two lieutenants have received the message “attack” from the general, therefore a consensus of 2/3 has been reached in the network and they will successfully attack the city, despite one lieutenant being a traitor. [22]

More complex example

A more complex problem is when there are three possible messages like “attack”, “retreat” and “wait” and the general is the traitor. He sends every lieutenant a different message. Each lieutenant receives the message and sends it to the other lieutenants to confirm the plan. Each lieutenant then knows that there have been different plans sent to each of them and therefore they realize that they received malicious information from the commander. Consequently, they reach a 3/3 consensus to retreat (default command). [22]

3.2.4 Replicated state machine

State Machine Replication is a fundamental approach in distributed computing for building fault-tolerant systems. It is a general method for managing replication across multiple servers and is widely used to compose a set of unreliable hosts into a single reliable service. Therefore, programmers can treat this service as a single system. In order to work, it requires the same operations to be received in the same order by each state machine. This can be achieved by distributed consensus. Therefore, it is important for the consensus algorithms that will be mentioned later. [33]

3.2.4.1 How do replicated state machines work?

A state machine consists of *state variables* and *commands*. State variables store the state (some information) of the system. Commands are a set of inputs. The state machine uses those commands to generate an output and an updated state. It does this by using a *transition function*. A basic description of the function of a state machine is as follows:

```
1    state = init
2    log = []
3    while true:
4        on receiving cmd from a client:
5            log.append(cmd)
6            state, output = apply(cmd, state)
7            send output to the client
```

Listing 3.1: Basic state machine function [18]

Here, the cmd is the input from the client. First, it appends the input to a log and then it executes the cmd by applying the *transition function* “apply” to the state. After that it returns the result “output” back to the client. [33]

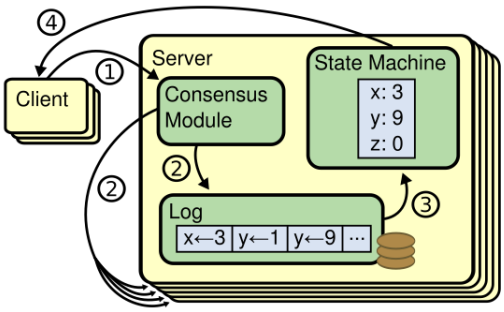


Figure 3.2: State machine architecture [31]

"The consensus algorithm manages a replicated log containing state machine commands from clients. The state machines process identical sequences of commands from the logs, so they produce the same outputs." [31] (Figure 3.2).

Fault-tolerant state machines An *f* fault-tolerant state machine can easily be implemented by replicating the state machine and running the replica on each of the processors in a distributed system. Provided that each replica is being run by a non-faulty processor in the same initial state and executes the same request in the same order, then each state machine will produce the same output with the same steps. [33] As already discussed in "Byzantine Generals Problem", a distributed system can experience Byzantine failures. To prevent such Byzantine faults, a fault-tolerant state machine must have at least 2f+1 replicas, and the output of the ensemble is the output produced by the majority of the state machine replicas. The reason for this is that the majority of the state machine replicas remain correct, after as many as f failures with 2f+1 replicas. [33]

3.2.5 Which consensus algorithm fits the needs for our system?

In this section I will discuss the two most widely used consensus algorithms to date, Paxos and Raft, explain how they work and describe their differences. I will also compare which one works better for our system. Because Paxos is not a single algorithm but a family of algorithms, *multi-decree Paxos*, which corresponds more closely to how Paxos is used today, will be compared to in this paper and mentioned as Paxos. [15]

Both algorithms are very similar with only a few key differences. Therefore, I will first explain the basics of how they work.

3.2.5.1 How do Paxos and Raft implement consensus?

They work by first electing a distinguished leader, who is completely responsible for managing the replicated log. The leader basically accepts client requests (log entries), replicates them to the other servers in the cluster and tells those servers when it is safe to apply log entries to their state machines. [31]

Basics Let's say we have five servers in a Raft cluster, which means that the system is able to tolerate two failures (as mentioned in "fault-tolerant state machines" 3.2.4.1). A server must be in one of the three states: *Leader, follower or candidate*. There is exactly one leader in the cluster while the other servers are followers. Followers just respond to requests from the leader or candidates, they do not issue them on their own. If a client speaks to a follower, the follower redirects the request to the leader, since only the leader can handle the client request. The candidate state is used to elect a new leader. Time is divided into *terms*. A term begins with a leader election, where candidates attempt to become leaders. A candidate

server stays as the leader for the rest of the term after it wins the election. With both algorithms, the cluster communicates over remote procedure calls (RPCs). Typically, there are two types of RPCs: [31] [15]

- **RequestVote RPCs** are used in an election by candidates.
- **AppendEntries RPCs** are used by leaders for log replication and heartbeats, which will be covered later.

Summarized, a node can be in the following states: [15] [31]

- **Follower** A passive state. A follower is only responsible for replying to RPCs.
- **Candidate** An active state. A candidate is trying to become the leader using RequestVotes RPCs.
- **Leader** An active state. The leader "is responsible for adding operations to the replicated log using AppendEntries RPCs". [15]

Algorithms lifecycle For better understandability, here is a step by step explanation of the cluster from startup to responding to clients. [31]

1. All nodes start in the follower state
   Leader election begins
2. If a follower does not get a life sign from the leader, he can become a candidate
3. The candidate requests votes from other nodes in the cluster through RequestVote RPCs.
4. The nodes will reply with their vote
5. The candidate becomes the leader if the majority of nodes voted for him.
   Leader election completes. Log replication begins
6. All changes to the system now go through the leader
7. A change gets stored in the leader's log, though because it is uncommitted, it will not update the leader's value
8. The leader replicates it to the followers
9. If the majority of the followers have written the entry, the entry gets committed on the leader
10. The leader notifies the followers that the entry is committed, therefore the cluster has now come to a consensus about the system state

This process continues until the leader fails and a new leader needs to be established. Paxos and Raft approach this process differently.[15] This leads me to the key differences between those two algorithms.

3.2.5.2 Key differences between Paxos and Raft

Leader election Both use heartbeats to trigger a leader election. A follower stays in his state as long as he receives valid heartbeats (empty AppendEntries RPCs) from a leader or a candidate. If a follower does not receive heartbeats anymore, it times out, called *election timeout*. The following Figure 3.3 depicts the nodes' transitions between their states, which will be covered for both algorithms in more detail now.

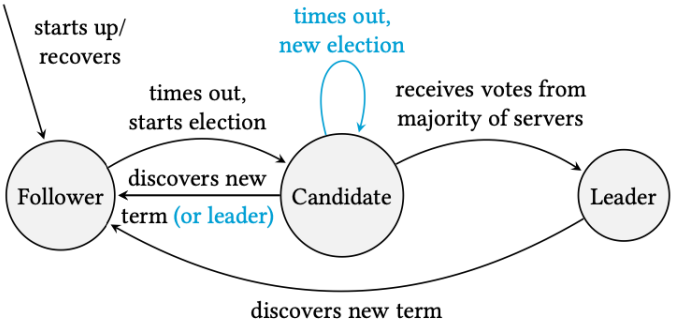


Figure 3.3: "State transition between the server states for Paxos & Raft. The transition in blue are specific to Raft." [15]

Paxos. After a follower times out, it becomes a candidate, updates its term to the next term "such that $t \bmod n = s$ where t is the next term, n is the number of servers and s is the candidate's server-id."[15] Then the candidate sends out RequestVote RPCs to the other nodes, which include the candidate's new term and the commit index. When a node receives the RequestVote RPC, it checks if the candidate's term is greater than its own and then responds positively. The candidate also sends any log entries that it has in its log subsequent to the candidate's commit index in the response. The candidate becomes the leader after receiving the majority of positive RequestVote responses, but first, it has to update its log and ensure it includes all committed entries. It does it by going through every index after the commit index, reviews the log entries it has received and updates its own log with the entry and new term for the index. In case the candidate received multiple log entries for the same index, it updates its log with the entry from the greatest term and the new term. After that, the candidate becomes the leader and begins replicating its log to the other nodes. [15] [19]

Raft. After a follower times out, it then begins a new election and transitions to the candidate state, while incrementing its current term. Following that, it sends out RequestVote RPCs to other nodes, which include the last log term and index. When a node receives the RequestVote RPC, it checks if the candidate's term is greater than or equal to its own, the candidate's log is at least as up-to-date as its own and it has not yet voted in this term. If all the conditions above as satisfied, it responds positively. The candidate wins if the majority of the cluster voted for him. If in the meantime, another server establishes itself as the leader, the candidate goes back to the follower state. As there may be multiple candidates in an election, a split vote can occur. In that case, the candidate times out again and starts a new election with the next term. To minimize split votes, the election timeout is randomized for each server in the cluster. [31] [15]

Log replication Once a leader is elected, it starts handling requests, which contain a command to be executed by the replicated state machines. The command gets appended to the leader's log and AppendEntries RPCs get sent out to other servers to replicate the log. Once the majority of the servers replicated

the log, the leader commits the log to the state machine and tells the other servers to do so either. If followers do not append entries because they crash or experience packet loss, the leader retries AppendEntries RPCs indefinitely until all followers eventually store all log entries. [31]

Performance and Understandability. The official Raft paper "In Search of an Understandable Consensus Algorithm", mentions: "Raft is a consensus algorithm for managing a replicated log. It produces a result equivalent to (multi-)Paxos, and it is as efficient as Paxos, but its structure is different from Paxos; this makes Raft more understandable than Paxos and also provides a better foundation for building practical systems." [31]

Understandability. A big advantage of Raft for programmers is that understandability was a major feature in its development, as mentioned in the paragraph above. The reason for this was that Paxos is relatively hard to understand and moreover implement. Raft also provides a better foundation for building practical systems because there is no widely agreed-upon algorithm for multi-Paxos and it is more of a "do it yourself" algorithm. [31] The original paper that described Paxos, "The Part-Time Parliament" by Leslie Lamport [20] got rejected three times before it was accepted because it was not understood. [21] The developers of Raft conducted an experimental study comparing the understandability of those two algorithms "using upper-level undergraduate and graduate students in an Advanced Operating Systems course at Stanford University and a Distributed Computing course at U.C. Berkeley." [31] The outcome of this experiment was that after watching a lecture for each algorithm, participants scored on average 4.9 points higher on the Raft quiz than on the Paxos quiz (out of 60 possible points). Furthermore, in a survey after the quizzes, 33 of 41 participants reported that Raft would be easier to implement and explain. "Using a 5-point scale, participants were asked which algorithm they felt would be easier to implement in a functioning, correct, and efficient system, and which would be easier to explain to a CS graduate student." (Figure 3.4) [31]

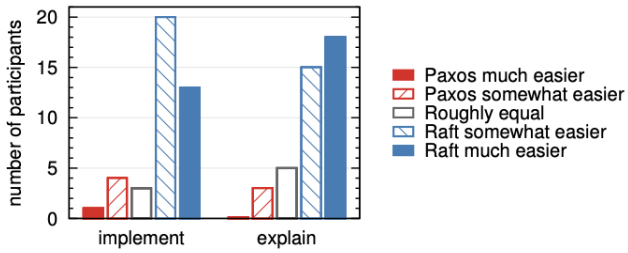


Figure 3.4: Survey comparing Paxos and Raft in terms of understandability. [15]

Performance. An advantage of Paxos is that leader elections do not feature split votes, which should theoretically make it more efficient. However, Raft's implementation of leader election makes it more lightweight because Raft only allows a candidate to become a leader if he has an up-to-date log. Therefore, it does not send log entries during the leader election, while with Paxos every RequestVote response includes the follower's log entries after the candidate's commit index. Also, in both algorithms, the new leader copies its log to all other servers. Here, Paxos exchanges more log entries because "a log entry may have been given a new term by the leader and thus the leader may send another copy of the log entry to a server which already has a copy."[15] Raft does not do that because each log entry keeps the same term throughout its time in the log. "Overall, compared to Paxos, Raft's approach to leader election is surprisingly efficient for such a simple approach."[15]

	Paxos	**Raft**
How does it ensure that each term has at most one leader?	A server s can only be a candidate in a term t if t mod n = s. There will only be one candidate per term so only one leader per term	A follower can become a candidate in any term. Each follower will only vote for one candidate per term, so only one candidte can get a majority of votes and become a leader.
How does it ensure that a new leader's log contains all committed log entries?	Each RequestVote reply includes the follower's log entries. Once a candidate has received RequestVote responses from a majority of followers, it adds the entries with the highest term to its log	A vote is granted only if a candidate's log is at least as up-to-date as the follower's. This ensures that a candidate only becomes a leader if its log is at least as up-to-date as a majority of followers.
How does it ensure that leaders safely commit log entries from previous terms?	Log entries from previous terms are added to the leader's log with the leader's term. The leader then replicates the log entries as if they were from the leader's term.	The leader replicates the log entries tot the other servers without changing the term. The leader cannot consider these entries committed until it has replicated a subsequent log entry from its own term.

Table 3.1: Summary of the differences between Paxos and Raft. [15]

3.2.5.3 Paxos or Raft?

Finally, a decision has to be made on which consensus algorithm to use for our project. This has multiple reasons. Performance does not play a leading role in this decision, since no algorithm performs outstandingly better than the other. The main factor to consider here is understandability. As mentioned in the above paragraph, Raft is easier to understand as well as to implement than Paxos. Furthermore, contrary to Paxos, Raft has a solid foundation for building practical systems, which in our case, is a huge advantage. Based on the facts mentioned above, the final decision on which consensus algorithm to use is **Raft**.

3.2.6 Conclusion

Consensus algorithms are an important part of distributed systems. They manage communication and replication between distributed parts of the system. With the help of the investigation, an answer to the research question, *What are consensus algorithms and how do they work? Which consensus algorithms are suitable for building a consistent and fault-tolerant network?*, could be found. The objective was achieved by analyzing and comparing the two most widely used consensus algorithms to date.

Glossary

actuator "An actuator is a component of a machine that is responsible for moving and controlling a mechanism or system, for example by opening a valve. In simple terms, it is a "mover"." [2]. 24

aggregations "Data aggregation is the process of gathering and expressing data in a summary to get more information about particular groups based on specific conditions." [9]. 24

algorithm "a procedure for solving a problem in a finite number of steps that frequently involves repetition of an operation." [**wiki-algorithm**]. 24

Apache Spark "Apache Spark is a fast, reliable, and fault-tolerant distributed computing framework for large-scale data processing." [24]. 24

Apache Storm "Apache Storm provides a scalable, fault-tolerant platform for implementing real-time data processing applications" [38]. 24

Application Programming Interface "An application programming interface (API) is a computing interface that defines interactions between multiple software intermediaries. It defines the kinds of calls or requests that can be made, how to make them, the data formats that should be used, the conventions to follow, etc."[3] . 24

autonomous "Autonomous things, abbreviated AuT, or the Internet of autonomous things, abbreviated as IoAT, is an emerging term for the technological developments that are expected to bring computers into the physical environment as autonomous entities without human direction, freely moving and interacting with humans and other objects." [4]. 24

bare-metal "A bare metal environment is a computer system or network in which a virtual machine is installed directly on hardware rather than within the host operating system (OS). The term "bare metal" refers to a hard disk, the usual medium on which a computer's OS is installed."[5]. 24

clusters "A cluster is a set of networked computers." [12]. 24

DAGs "directed acyclic graphs, a kind of graph that means that it doesn't form a circular loop" [10]. 24

Data API-Service the term Data API-Service is used to describe services which only purpose is to gather information from public Data-APIs and send them to other services. 24

data warehouse "Data warehouse applications maintain large data sets and can be mined for analytics." [25]. 24, 26

DNS "The Domain Name System (DNS) is a hierarchical and decentralized naming system for computers, services, or other resources connected to the Internet or a private network." [8]. 24

Docker "Docker is a software platform for building applications based on containers — small and lightweight execution environments that make shared use of the operating system kernel but otherwise run in isolation from one another"[46]. 24

entities "a thing with distinct and independent existence." [11]. 24

GoLang "Go is an open source programming language that makes it easy to build simple, reliable, and efficient software."[47]. 24

Hadoop MapReduce "Apache Hadoop MapReduce is an implementation of the MapReduce programming model that provides an embarrassingly parallel map phase where the data is split into subsets to be processed, followed by a reduce phase where the output of the map phase is aggregated to yield the desired result." [38]. 24

health check "Health checks are basically endpoints provided by a service to check whether the service is running properly."[13]. 24

Hive "Apache Hive is a data warehouse system for Hadoop that enables data summarization, querying, and analysis by using HiveQL (a query language similar to SQL)." [38]. 24

horizontally scaling "(...) systems scale by adding more machines (...)" [26]. 24

ip A unique address consisting of four numbers separated by dots for identifying devices connected to a network.[30]. 24

JSON "JavaScript Object Notation is a schema-less, text-based representation of structured data that is based on key-value pairs and ordered lists. Although JSON is derived from JavaScript, it is supported either natively or through libraries in most major programming languages."[48]. 24

Kafka Kafka is a message queuing system. [26]. 24

Kubernetes "Kubernetes is a portable, extensible, open-source platform for managing containerized workloads and services, that facilitates both declarative configuration and automation. It has a large, rapidly growing ecosystem. Kubernetes services, support, and tools are widely available."[49]. 24

legacy "A legacy system is outdated computing software and/or hardware that is still in use. The system still meets the needs it was originally designed for, but doesn't allow for growth. What a legacy system does now for the company is all it will ever do. A legacy system's older technology won't allow it to interact with newer systems. "[45]. 24

message queuing system "A messaging/queuing system provides a way to send and consume messages between processes in a fault-tolerant and asynchronous manner. A message queue is a key component for doing realtime processing." [26]. 24

metadata "Metadata is "data that provides information about other data". In other words, it is "data about data". Many distinct types of metadata exist, including descriptive metadata, structural metadata, administrative metadata, reference metadata and statistical metadata." [27]. 24

micro controller "A micro controller is a compact integrated circuit designed to govern a specific operation in an embedded system." [28] . 24

OSI-Layers "Introduced in 1978, the ISO Open Systems Interconnection (OSI) Reference model describes networking as "a series of protocol layers with a specific set of functions allocated to each layer. Each layer offers specific services to higher layers while shielding these layers from the details of how the services are implemented. A well-defined interface between each pair of adjacent layers defines the services offered by the lower layer to the higher one and how those services are accessed." [29].. 24

overhead "In computer science, overhead is any combination of excess or indirect computation time, memory, bandwidth, or other resources that are required to perform a specific task. It is a special case of engineering overhead. Overhead can be a deciding factor in software design, with regard to structure, error correction, and feature inclusion. Examples of computing overhead may be found in functional programming, data transfer, and data structures." [36]. 24

persisted "the data is stored for the long term" [38]. 24

Protocol Buffers "Protocol buffers are Google's language-neutral, platform-neutral, extensible mechanism for serializing structured data – think XML, but smaller, faster, and simpler. "[43]. 24

Proxy "A proxy service is an intermediary role played by software or a dedicated computer system between an endpoint device and a client which is requesting the service. The proxy service may exist on the same machine or on a separate server. The proxy service enables the client to connect to a different server and provides easy access to services like Web pages, connections or files."[32]. 24

Remote Procedure Call "The idea of remote procedure calls (hereinafter called RPC) is quite simple. It is based on the observation that procedure calls are a well-known and well- understood mechanism for transfer of control and data within a program running on a single computer. Therefore, it is proposed that this same mechanism be extended to provide for transfer of control and data across a communication network. When a remote procedure is invoked, the calling environment is suspended, the parameters are passed across the network to the environment where the procedure is to execute (which we will refer to as the callee), and the desired procedure is executed there. When the procedure finishes and produces its results, the results are passed backed to the calling environment, where execution resumes as if returning from a simple single-machine call. While the calling environment is suspended, other processes on that machine may (possibly) still execute (depending on the details of the parallelism of that environment and the RPC implementation)." [6]. 24

Replication Factor "Replication is the process of copying data from a central database to one or more databases." [16]. 24

REpresentational State Transfer "REST stands for REpresentational State Transfer. A RESTful web application exposes information about itself in the form of information about its resources. It also enables the client to take actions on those resources, such as create new resources (i.e. create a new user) or change existing resources (i.e. edit a post)."[50]. 24

scaled vertically "With vertical scaling (a.k.a. "scaling up"), you're adding more power to your existing machine. In horizontal scaling (a.k.a. "scaling out"), you get the additional resources into your system by adding more machines to your network, sharing the processing and memory workload across multiple devices."[42]. 24

scaling the system vertically "(...) trying to scale by adding a better machine (...)" [26]. 24

Sensor-Service The term sensor-services is used to describe services which only purpose is to gather data from sensors and send them to other services. 24

serialization "In computing, serialization (US spelling) or serialisation (UK spelling) is the process of translating a data structure or object state into a format that can be stored (for example, in a file or memory data buffer) or transmitted (for example, across a computer network) and reconstructed later (possibly in a different computer environment)." [34]. 24

service mesh "In software architecture, a service mesh is a dedicated infrastructure layer for facilitating service-to-service communications between microservices, often using a sidecar proxy." [35]. 24

SOA "Service-oriented architecture was largely created as a response to traditional, monolithic approaches to building applications. SOA breaks up the components required for applications into separate service modules that communicate with one another to meet specific business objectives. Each module is considerably smaller than a monolithic application, and can be deployed to serve different purposes in an enterprise."[23]. 24

SOAP "SOAP is an acronym for Simple Object Access Protocol. It is an XML-based messaging protocol for exchanging information among computers. SOAP is an application of the XML specification."[51]. 24

stream processing "Stream processing is the discipline and related set of techniques used to extract information from unbounded data" [24]. 24

syt "Als Systemtechnik bezeichnet man verschiedene Aufbau- und Verbindungstechniken, aber auch eine Fachrichtung der Ingenieurwissenschaften. Er bedeutet in der Unterscheidung zu den Mikrotechnologien die Verbindung verschiedener einzelner Module eines Systems und deren Konzeption." [**wiki-syt**]. 24

TCP "The Transmission Control Protocol (TCP) is one of the main protocols of the Internet protocol suite. It originated in the initial network implementation in which it complemented the Internet Protocol (IP). Therefore, the entire suite is commonly referred to as TCP/IP. TCP provides reliable, ordered, and error-checked delivery of a stream of octets (bytes) between applications running on hosts communicating via an IP network. "[40]. 24

telemetry collection "The collection and analysis of telemetry data from user's devices is routinely performed by many software companies. Telemetry collection leads to improved user experience but poses significant risks to users' privacy. Locally differentially private algorithms have recently emerged as the main tool that allows data collectors to estimate various population statistics, while preserving privacy."[44]. 24

tightly coupled "Connected software services are, broadly speaking, either more tightly-coupled or more loosely-coupled. Tight Coupling is the idea of binding resources to specific purposes and functions.

Tightly-coupled components may bind each resource to a particular use case, a specific interface, or a specific frontend.In essence, a tightly coupled system is purpose-built, and every custom deviation from the standard comes with its own resources and integrations. This is a simple way to build data relationships, and is beneficial in terms of understanding, relying upon, and interacting with said information. However, Tight Coupling brings clear losses to software extensibility and scalability."[39]. 24

times out "a situation in which a computer program stops working because an activity has taken too long"[1]. 21, 24

unbounded data "A type of dataset that is infinite in size (at least theoretically)" [24]. 24, 28

XML "XML stands for eXtensible Markup Language. XML is a markup language much like HTML. XML was designed to store and transport data. XML was designed to be self-descriptive"[17]. 24

Acronyms

TGM Technologisches Gewerbemuseum. 24

Bibliography

[1] 25.11.2020. Cambridge Dictionary. URL: https://dictionary.cambridge.org/de/worterbuch/englisch/timeout.

[2] *Actuator*. 2020-12-21. URL: https://www.thomasnet.com/articles/pumps-valves-accessories/types-of-actuators/,https://www.ekci.com/a-great-combination-pneumatic-actuator-timer-valves-and-indicators-ezp-69.html.

[3] *API*. 2020. URL: https://en.wikipedia.org/wiki/API.

[4] *Autonomous things*. 2020-12-21. URL: https://web.archive.org/web/20170419003202/http://cognitiveworld.com/articles/the-internet-of-autonomous-things-162.

[5] *bare metal environment*. 2006. URL: https://searchservervirtualization.techtarget.com/definition/bare-metal-environment.

[6] Andrew D Birrell and Bruce Jay Nelson. *Implementing Remote Procedure Calls*. Tech. rep. 1984. URL: http://www.cs.cmu.edu/{~}dga/15-712/F07/papers/birrell842.pdf.

[7] Tree Bressen. *Consensus Decision-Making: What, Why, How*. Tech. rep. URL: http://treegroup.info..

[8] *Domain Name System*. 2020-12-21. URL: https://tools.ietf.org/html/rfc1034,https://tools.ietf.org/html/rfc781.

[9] Dayong Du. *Apache Hive Essentials: Essential Techniques to Help You Process, and Get Unique Insights from, Big Data, 2nd Edition - Technische Universität Wien*. 2018. ISBN: 9781788995092. URL: https://catalogplus.tuwien.at/primo-explore/fulldisplay?docid=TN_cdi_proquest_ebookcentral_EBC5446045&context=PC&vid=UTW&lang=de_DE&search_scope=UTW&adaptor=primo_central_multiple_fe&tab=default_tab&query=any,contains,Big%20Data%20hive&offset=0.

[10] Irfan Elahi. *Scala Programming for Big Data Analytics*. 1st ed. Apress L. P, 2019. ISBN: 1484248090. DOI: 10.1007/978-1-4842-4810-2.

[11] *Entities definition*. 2020-11-26. URL: https://www.google.com/search?q=entities+definition&rlz=1C1CHBF_deAT831AT831&oq=Entities+def&aqs=chrome.0.0j69i57j0i22i30l6.4628j1j4&sourceid=chrome&ie=UTF-8.

[12] Nader Gemayel. "Analyzing Google File System and Hadoop Distributed File System". In: (2016). DOI: 10.3923/rjit.2016.66.74. URL: https://scialert.net/abstract/?doi=rjit.2016.66.74.

[13] *Health Checks for Microservices*. 2018. URL: https://dzone.com/articles/monitoring-microservices-with-health-checks.

[14] Parikshit Hooda. *Comparison - Centralized, Decentralized and Distributed Systems - GeeksforGeeks*. URL: https://www.geeksforgeeks.org/comparison-centralized-decentralized-and-distributed-systems/ (visited on 11/23/2020).

[15] Heidi Howard and Richard Mortier. "Paxos vs Raft". eng. In: *Proceedings of the 7th Workshop on principles and practice of consistency for distributed data*. PaPoC '20. ACM, pp. 1–9. ISBN: 9781450375245. DOI: 10.1145/3380787.3393681.

[16] IBM. *What is database replication?* URL: https://www.ibm.com/support/knowledgecenter/SSXW43{_}8.9.13/com.ibm.i2.ibase.admin.doc/what{_}is{_}database{_}replication.html.

[17] *Introduction to XML*. 2020. URL: https://www.w3schools.com/xml/xml_whatis.asp.

[18] Nayak Kartik and Abraham Ittai. *Consensus for State Machine Replication*. URL: https://decentralizedthoughts.github.io/2019-10-15-consensus-for-state-machine-replication/ (visited on 11/23/2020).

[19] Leslie Lamport. *Paxos Made Simple*. Tech. rep. 2001.

[20] Leslie Lamport. "The part-time parliament". eng. In: *ACM transactions on computer systems* 16.2 (), pp. 133–169. ISSN: 0734-2071. DOI: 10.1145/279227.279229.

[21] Leslie Lamport. *The Writings of Leslie Lamport*. URL: https://lamport.azurewebsites.net/pubs/pubs.html{\#}lamport-paxos (visited on 11/23/2020).

[22] Leslie Lamport, Robert Shostak, and Marshall Pease. "The Byzantine Generals Problem". eng. In: *ACM transactions on programming languages and systems* 4.3 (), pp. 382–401. ISSN: 0164-0925. DOI: 10.1145/357172.357176.

[23] *MA vs SOA*. URL: https://www.talend.com/resources/microservices-vs-soa/.

[24] Gerard Maas and Francois Garillot. *Stream Processing with Apache Spark*. O'Reilly Media, Incorporated, 2019. ISBN: 1491944242.

[25] Zaigham Mahmood. *Data science and big data computing: Frameworks and methodologies*. Springer International Publishing, 2016, pp. 1–319. ISBN: 9783319318615. DOI: 10.1007/978-3-319-31861-5.

[26] Nathan Marz and James Warren. *Big data : principles and best practices of scalable real-time data systems*. Shelter Island, NY : Manning, 2015. ISBN: 9781617290343. URL: https://permalink.catalogplus.tuwien.at/AC12280866.

[27] *Metadata*. 2020-12-21. URL: https://www.merriam-webster.com/dictionary/metadata, https://marciazeng.slis.kent.edu/metadatabasics/types.htm, https://stats.oecd.org/glossary/detail.asp?ID=7076.

[28] *microcontroller (MCU)*. 2021. URL: https://internetofthingsagenda.techtarget.com/definition/microcontroller.

[29] Microsoft. *Windows Network Architecture and the OSI Model*. URL: https://docs.microsoft.com/en-us/windows-hardware/drivers/network/windows-network-architecture-and-the-osi-model.

[30] Stefan Nilsson and Gunnar Karlsson. *IP-Address Lookup Using LC-Tries*. 1998.

[31] Diego Ongaro and John Ousterhout. *In Search of an Understandable Consensus Algorithm (Extended Version)*. Tech. rep.

[32] *Proxy Service*. 2020. URL: https://www.techopedia.com/definition/31705/proxy-service.

[33] Fred B Schneider. *The State Machine Approach: * A Tutorial*. Tech. rep.

[34] *Serialization*. 2020-12-21. URL: https://web.archive.org/web/20150405013606/http://isocpp.org/wiki/faq/serialization,http://erights.org/data/serial/jhu-paper/intro.html.

[35] *Service mesh*. 2020-12-21. URL: https://en.wikipedia.org/wiki/Service_mesh.

[36] Daniel J Sorin. *Caches and Memory Hierarchies*. 2020-12-21. URL: http://people.ee.duke.edu/~sorin/prior-courses/ece152-spring2009/lectures/6.1.3-caches.pdf.

[37] Andrew S Tanenbaum and Maarten van Steen. *Distributed systems : principles and paradigms*. eng. 2. ed., Pearson new internat. ed.. Always learning. Harlow: Pearson, 2014. ISBN: 1292025522.

[38] Zoiner Tejada. *Mastering Azure Analytics*. O'Reilly Media, Incorporated, 2017. ISBN: 9781491956656.

[39] *The Difference Between Tight Coupling and Loose Coupling*. 2020. URL: https://nordicapis.com/the-difference-between-tight-coupling-and-loose-coupling/.

[40] *Transmission Control Protocol*. 2020. URL: https://en.wikipedia.org/wiki/Transmission_Control_Protocol.

[41] J Turek and D Shasha. "The many faces of consensus in distributed systems". eng. In: *Computer (Long Beach, Calif.)* 25.6 (), pp. 8–17. ISSN: 0018-9162. DOI: 10.1109/2.153253.

[42] *Vertical vs Horizontal Scalling*. 2018. URL: https://www.missioncloud.com/blog/horizontal-vs-vertical-scaling-which-is-right-for-your-app.

[43] *What are Protocol Buffers?* 2020. URL: https://developers.google.com/protocol-buffers.

[44] *What are Protocol Buffers?* 2020. URL: https://developers.google.com/protocol-buffers.

[45] *What is a legacy system?* 2020. URL: https://www.talend.com/resources/what-is-legacy-system/#:~:text=A%20legacy%20system%20is%20outdated,doesn't%20allow%20for%20growth.&text=A%20legacy%20system's%20older%20technology,to%20interact%20with%20newer%20systems..

[46] *What is Docker?* 2019. URL: https://www.infoworld.com/article/3204171/what-is-docker-the-spark-for-the-container-revolution.html.

[47] *What is GoLang*. 2020. URL: https://golang.org/.

[48] *What is JSON?* 2019. URL: https://www.infoworld.com/article/3222851/what-is-json-a-better-format-for-data-exchange.html.

[49] *What is Kubernetes*. 2020. URL: https://kubernetes.io/docs/concepts/overview/what-is-kubernetes/.

[50] *What is Rest?* 2017. URL: https://medium.com/extend/what-is-rest-a-simple-explanation-for-beginners-part-1-introduction-b4a072f8740f.

[51] *What is SOAP?* 2020. URL: https://www.tutorialspoint.com/soap/what_is_soap.htm.

List of Figures

3.1 Centralized system (left) and distributed system (right) [14] 15
3.2 State machine architecture [31] 19
3.3 “State transition between the server states for Paxos & Raft. The transition in blue are specific to Raft.” [15] 21
3.4 Survey comparing Paxos and Raft in terms of understandability. [15] 22

List of Tables

3.1 Summary of the differences between Paxos and Raft. [15] 23

List of Listings

3.1 Basic state machine function [18] 18